

SMC Fabric — Internal Architecture

SMC Fabric — Internal Architecture

1. Input Stage

- 1.1 ID Width Normalisation
- 1.2 Log AXI-Lite Acceptance
- 1.3 Alias Remap
- 1.4 Arbitration (Mux 4 → 1)
- 1.5 Local / Global Aperture Split
- 1.6 Address Truncation (56 → 32)
- 1.7 System Inbound Filter
- 1.8 SEP Address Truncation

2. Local Crossbar

- 2.1 Address Masking
- 2.2 Primary Crossbar Output Ports
- 2.3 Internal CSR Crossbar

3. Output Stage

- 3.1 Bypass Mode (`no_addr_remap = true` , default)
- 3.2 Full Remap Mode (`no_addr_remap = false`)
- 3.3 Outbound Filter

4. Routing Decision Summary

SMC Fabric — Internal Architecture

This document describes each logical sub-block of `smc_fabric` and its LT modelling strategy.

1. Input Stage

Handles all traffic originating from SMC-internal initiators (CPU MMIO, JTAG, DMA/Zeroer, Log) and from the external `sys_axi_in` / `sep_axi_in` ports.

1.1 ID Width Normalisation

AXI ID widths vary per initiator port. In the LT model, ID width is irrelevant — `smc_axi_extension.axi_id` carries the raw hart or transaction ID and passes through unchanged. No conversion logic is needed in the SystemC model.

1.2 Log AXI-Lite Acceptance

The `log_in` target socket accepts standard `tlm_generic_payload` objects just like any other target socket. The LT model routes it identically to the MMIO and JTAG paths after alias remap.

1.3 Alias Remap

All four internal initiators (MMIO, JTAG, Log, DMA/Zeroer) share a single 8-entry alias remap table, populated from the `ALIAS_REMAP` CSRs.

Algorithm per transaction: 1. Walk entries 0–7 in order; skip entries where `valid = false`. 2. First match where `region_start ≤ addr < region_end`: apply `addr_out = addr_in + offset`. Set the cacheable hint in `smc_axi_extension` if `cacheable = true`. 3. No match: address passes through unchanged.

```
struct alias_region {
    uint64_t start;    // inclusive
    uint64_t end;      // exclusive
    int64_t  offset;   // signed
    bool     cacheable;
    bool     valid;
};

uint64_t apply_alias_remap(uint64_t addr,
                          const std::array<alias_region, 8>& tbl)
{
    for (const auto& r : tbl) {
        if (r.valid && addr >= r.start && addr < r.end)
            return addr + static_cast<uint64_t>(r.offset);
    }
    return addr;
}
```

The table is rebuilt whenever a write arrives on `to_ar_ctrl`. After any rebuild, `invalidate_direct_mem_ptr(0, UINT64_MAX)` is called on all initiator sockets.

1.4 Arbitration (Mux 4 → 1)

Four alias-remapped internal initiators merge into a single stream before the local/global split. In LT this is a no-op: `b_transport` is cooperatively serialised by the SystemC scheduler — whichever initiator calls first proceeds immediately.

1.5 Local / Global Aperture Split

After alias remap, each transaction is classified as *local* or *outbound*:

```
bool is_local(uint64_t addr) const {
    return (addr >= local_base_ && addr < local_base_ + region_size_) ||
           (addr >= global_base_ && addr < global_base_ + region_size_);
}
```

- **Local** → address truncated to 32 bits → forwarded to the local crossbar.
- **Outbound** → forwarded to the output stage.

1.6 Address Truncation (56 → 32)

All local-bound transactions have their address narrowed before entering the local crossbar:

```
uint32_t to_local_addr(uint64_t addr, uint64_t local_base) {
    // Force upper 7 bits to match local_base[31:25] so all paths
    // into the local crossbar decode consistently.
    return (static_cast<uint32_t>(local_base) & 0xFE000000u)
           | (static_cast<uint32_t>(addr) & 0x01FFFFFFu);
}
```

1.7 System Inbound Filter

Applied only to `sys_axi_in` traffic, before it reaches the local crossbar. Policy: **deny by default** (allow-list). 16 entries.

Per-transaction decision:

```
bool inbound_filter_allow(uint64_t addr, uint8_t src_id, bool ns) const {
    for (const auto& e : inbound_filter_) {
        if (!e.enabled) continue;
        if (addr < e.addr_lo || addr >= e.addr_hi) continue;
        if (e.src_en && (src_id & e.src_mask) != e.src_id) continue;
        if (e.ns_en && ns != e.ns_req) continue;
        return e.allow;
    }
    return false; // deny by default
}
```

Deny response: `TLM_ADDRESS_ERROR_RESPONSE`, read data filled with `0xBADCAB1E`.

1.8 SEP Address Truncation

`sep_axi_in` traffic bypasses all remap and filter logic. The address is simply truncated to 32 bits using the same `to_local_addr()` helper, then forwarded to the local crossbar. SEP traffic never reaches the outbound path.

2. Local Crossbar

Routes traffic from three input paths (system inbound, SEP inbound, SMC-local from the input stage) to multiple local targets.

2.1 Address Masking

Before any decode, every input address is masked so its upper 7 bits match `local_base_addr[31:25]`. This ensures consistent decoding regardless of whether the transaction arrived via the local alias, global alias, or the system/SEP ports.

```
uint32_t mask_addr(uint32_t addr) const {  
    return (static_cast<uint32_t>(cfg_.local_base_addr) & 0xFE000000u)  
        | (addr & 0x01FFFFFFu);  
}
```

2.2 Primary Crossbar Output Ports

Address-range decode in `route_local()` maps a 32-bit post-masked address to one of the following sockets:

Socket	Description
<code>to_front_port</code>	General AXI4 local targets: SRAM, ROM, BEU, PLIC, CLINT, ...
<code>to_data_accel_ctrl</code>	DMA engine + memory zeroer control registers
<code>to_periph</code>	AXI4-Lite 32-bit peripheral crossbar
<code>to_dfd_apb</code>	APB DFD registers
Internal CSR path	Feeds the internal CSR crossbar (see §2.3)

2.3 Internal CSR Crossbar

A second decode level fans out the internal CSR address range to the fabric's own configuration sockets:

Socket	Description
<code>to_cpu_ctrl</code>	CPU control registers
<code>to_aR_ctrl</code>	Alias remap configuration
<code>to_mR_ctrl</code>	M-mode output remap configuration
<code>to_xR_ctrl</code>	Xvisor output remap configuration
<code>to_inbound_filter_ctrl</code>	Inbound filter entries
<code>to_outbound_filter_ctrl</code>	Outbound filter entries
<code>to_mailbox</code>	Mailbox unit CSRs
<code>to_dft_csr</code>	DFT CSRs

Writes to the remap and filter sockets update the fabric's internal tables immediately and trigger `invalidate_all_dmi()`. Writes to the other sockets are forwarded to the bound downstream modules.

3. Output Stage

Handles the outbound path to the system NoC. Activated only for transactions that fail the `is_local()` check.

3.1 Bypass Mode (`no_addr_remap = true`, default)

When bypass mode is active the address and `smc_axi_extension` fields pass through unchanged. The transaction goes directly to the outbound filter and then to `output_axi`.

3.2 Full Remap Mode (`no_addr_remap = false`)

3.2.1 Address Classification

Each outbound transaction is classified into one of three paths:

Path	Condition	<code>source_id</code> override
plain	Not M-mode and not Xvisor	<code>SMC_SRC_ID</code>
M-mode	<code>addr ∈ [LOCAL/GLOBAL_BASE + MMODE_REMAP_START, + MMODE_REMAP_SIZE)</code>	<code>MMODE_SRC_ID</code>
Xvisor	<code>addr ∈ [LOCAL/GLOBAL_BASE + XVISOR_REMAP_START, + XVISOR_REMAP_SIZE)</code>	<code>OTHERS_SRC_ID</code>

```
enum class outbound_path { plain, mmode, xvisor };

outbound_path classify_outbound(uint64_t addr) const {
    auto in_win = [&](uint64_t base, uint64_t start, uint64_t sz) {
        return addr >= base + start && addr < base + start + sz;
    };
    if (in_win(global_base_, MMODE_REMAP_START, MMODE_REMAP_SIZE) ||
        in_win(local_base_, MMODE_REMAP_START, MMODE_REMAP_SIZE))
        return outbound_path::mmode;
    if (in_win(global_base_, XVISOR_REMAP_START, XVISOR_REMAP_SIZE) ||
        in_win(local_base_, XVISOR_REMAP_START, XVISOR_REMAP_SIZE))
        return outbound_path::xvisor;
    return outbound_path::plain;
}
```

3.2.2 M-mode and Xvisor Remap

Both remap types use the same range-match + signed-offset logic as alias remap, applied to separate 8-entry tables populated from `MMODE_REMAP` and `XVISOR_REMAP` CSRs respectively.

```
uint64_t apply_output_remap(uint64_t addr,
                           const std::array<alias_region, 8>& table,
                           uint8_t src_id_override,
                           smc_axi_extension* ext) const
{
    for (const auto& r : table) {
        if (!r.valid) continue;
        if (addr >= r.start && addr < r.end) {
            if (ext) ext->source_id = src_id_override;
            return addr + static_cast<uint64_t>(r.offset);
        }
    }
    return addr;
}
```

3.3 Outbound Filter

Applied to all outbound traffic (both bypass and full-remap paths) immediately before `output_axi`. Policy: **allow by default** (deny-list). 16 entries.

```
bool outbound_filter_allow(uint64_t addr, uint8_t src_id, bool ns) const {
    for (const auto& e : outbound_filter_) {
        if (!e.enabled) continue;
        if (addr < e.addr_lo || addr >= e.addr_hi) continue;
        if (e.src_en && (src_id & e.src_mask) != e.src_id) continue;
        if (e.ns_en && ns != e.ns_req) continue;
        return e.allow;
    }
    return true; // allow by default
}
```

4. Routing Decision Summary

Incoming socket	Step 1	Step 2	Step 3	Destination
jtag_axi_in	alias remap	is_local? → yes	addr decode	to_front_port / to_data_accel_ctrl / to_periph / ...
jtag_axi_in	alias remap	is_local? → no	output remap + filter	output_axi
mmio_in	alias remap	is_local?	addr decode / output path	same as above
data_accel_in	alias remap	is_local?	addr decode / output path	same as above
log_in	alias remap	is_local?	addr decode / output path	same as above
sys_axi_in	inbound filter	pass → addr truncate	addr decode	local targets only
sep_axi_in	addr truncate	addr decode	—	local targets only

sys_axi_in and sep_axi_in traffic **never** reaches the outbound path.